# Accelerating High-Order Wavefront Sensing and Control Algorithms with FPGAs

M.Subhi Abo Rdan[a], Nicholas Belsten[a], Shanti Rao , Ahmad W Taka[a], Brandon Eickert[a], Kian Milani[b], Ewan S. Douglas[b], Leonid Pogorelyuk[c], and Kerri Cahoy[a]

[a]MIT Department of Aeronautics and Astronautics, Cambridge, MA, USA
[b]University of Arizona, Tucson, AZ, USA
[c]Rensselaer Polytechnic Institute, Troy, NY, USA

## ABSTRACT

Future space telescopes will rely on real-time wavefront sensing and control to achieve the high contrast necessary to directly image Earth-like exoplanets around Sun-like stars. The unique radiation and resource-constrained environment of space limits the use of high-performance computers that would be used for ground-based applications, and current generation radiation-hardened CPUs are unlikely to meet the computational demands of wavefront sensing and control on future missions like the Habitable Worlds Observatory (HWO). Although alternative system architectures such as the ground-in-the-loop configuration may be a viable solution for some latency requirements, telescope designs with wavefront control periods of less than about 10 seconds will necessitate on-board computing, as the light time delay precludes a ground-in-the-loop architecture. Other computationally intensive space missions, such as radar beam-forming or hyperspectral image processing, often take advantage of the unique capabilities of Field Programmable Gate Arrays (FPGAs), which are available in radiation-hardened and radiation-tolerant packages.

In this work, we accelerate key computational bottlenecks for high-order wavefront sensing and control (HOWFSC) via custom-designed FPGA compute kernels. We integrate these compute kernels into a demonstration application utilizing an AMD Zynq UltraScale+ Multi-Processor System-on-Chip (MPSoC). In particular, we accelerate Electric Field Conjugation (EFC) for optical control, matrix Fourier transforms (MFTs), and angular spectrum propagation for optical modeling. We demonstrate speed improvements of orders of magnitude, including a 360× speedup of 2D FFT over an implementation of Tukey's algorithm on the MPSoC Arm Cortex-A53 application processor and a 500× speedup in angular spectrum propagation—both critical operations for computing the Jacobian sensitivity matrix.

We discuss the new architectural constraints for further performance improvement within the FPGA design and evaluate new bottlenecks that have not yet been accelerated with the new FPGA kernels. Our demonstrated design could be easily ported to analogous radiation-hardened UltraScale components to support space missions like HWO. This new processing capability will support coronagraphs with larger pixel counts, larger DM counts, and shorter wavefront control periods, thereby improving overall instrument contrast and increasing the likelihood of detection of an Earth-like exoplanet.

**Keywords:** HOWFSC, Adaptive optics, FPGA, Accelerated Computing, Embedded systems, High-contrast imaging, Angular spectrum method, Fourier Optics

Direct correspondence to msubhi_a@mit.edu

# 1 INTRODUCTION

Adaptive Optics (AO) enhances imaging and light emission across a range of applications, including laser communication and astronomical imaging. Central to these systems is the ability to sense and correct wavefront aberrations. While many applications are adequately served by wavefront sensing at the pupil plane, others are highly sensitive to small non-common-path aberrations, which occur between the wavefront sensor and the final imaging plane. High-contrast imaging is one such application. In these cases, it is beneficial to use measurements from the imaging camera itself to determine the appropriate commands for the deformable mirror (DM) in the AO system. This technique, known as image plane wavefront sensing and control, is commonly employed to correct high-order aberrations and is therefore often referred to as High-Order Wavefront Sensing and Control (HOWFSC).

The conversion of amplitude information in the image plane to appropriate DM commands in other planes (such as the pupil plane) involves many-dimensional estimation and control. In the application of high-contrast imaging, we seek to create a region of very high contrast (a "dark hole"), so the desired control input is one that cancels out, or conjugates, the electric field in the image plane. The foundational algorithm to perform this control is Electric Field Conjugation (EFC). In EFC, a linearized model of the optical system relates every DM command to the change in the electric field states in the image. This linearized sensitivity is represented in matrix form—the Jacobian matrix. This Jacobian matrix contains as many rows as there are electric field states, which is proportional to the number of controlled pixels, with an additional factor of two to account for the real and imaginary components of the electric field, and possibly additional factors to account for multi-spectral HOWFSC or multiple polarizations. The Jacobian matrix contains as many columns as there are DM actuators.

One motivational mission for this work is the future planned telescope, the Habitable Worlds Observatory (HWO). One goal of HWO is to achieve the $10^{-10}$ contrast necessary to image Earth-like exoplanets around Sun-like stars. Previous works have discussed the significant computational challenges of multiple algorithms for HOWFSC as they relate to a mission like HWO. In this present work, we demonstrate accelerations of key numerical methods and full algorithms on a Multi-Processor System-on-Chip (MPSoC), which contains a Field Programmable Gate Array (FPGA) for acceleration.

In space mission design, it is conventionally held amongst mission planners that implementing computing on a CPU (Central Processing Unit) has the lowest development cost but the least performance. At the other end of that spectrum is the ASIC (Application-Specific Integrated Circuit), which has the highest development cost but the best performance. Then the FPGA is somewhere in the middle. The last few decades of microprocessor development have moved integrated circuit (IC) designs to smaller and smaller feature sizes, thereby improving the speed of CPUs (as well as FPGAs and ASICs), but also dramatically increasing the non-recurring costs of a fully custom ASIC design. Many modern space missions choose to leverage the strong capability of commercial components rather than undertaking the onerous challenges and costs of ASIC design. However, FPGAs are still used for many space applications, particularly in the domains of radar and real-time signal processing. In this work, we demonstrate the implementation of FPGA-accelerated HOWFSC methods. We quantify the speed-ups that are achievable with FPGAs compared to similar CPU implementations, and we identify those methods that lend themselves to FPGA acceleration, and those which do not.

Throughout this work, we have roughly sized our implementations to be relevant for a mission like HWO, but many other AO and HOWFSC applications would benefit from the adoption of the FPGA-acceleration methods which we demonstrate, including ground-based observatories.

The rest of this paper is organized as follows: we end this introduction with a brief survey of processors—particularly those which are suitable for the space environment. We next discuss the numerical composition of HOWFSC control algorithms in Section 2. In Section 3, we discuss the design of our FPGA acceleration kernels. The quantitative results of FPGA acceleration are reported and discussed in Section 4, and we discuss implications for high contrast imaging missions like HWO in Section 5.

### 1.1 Processor Survey

The performance demands of high-order wavefront sensing and control (HOWFSC) in future space telescopes, such as the Habitable Worlds Observatory (HWO), may exceed the capabilities of many traditional radiation-tolerant CPUs. Previous processor surveys[1][2] highlight that while CPUs continue to follow a predictable Moore's Law trajectory, they are fundamentally constrained by memory bandwidth for many applications, leading to multi-order-of-magnitude shortfalls between theoretical compute density and observed benchmark performance. In contrast, heterogeneous architectures like FPGAs demonstrate significantly higher compute density and memory throughput for dataflow-oriented tasks typical of HOWFSC. Figure 1 shows the relatively higher compute density of heterogeneous devices over time. Figure 2 shows that FPGA-based devices like the XQR Versal dominate the design space in both compute and memory performance metrics, often achieving up to 10× higher throughput than contemporaneous space CPUs.

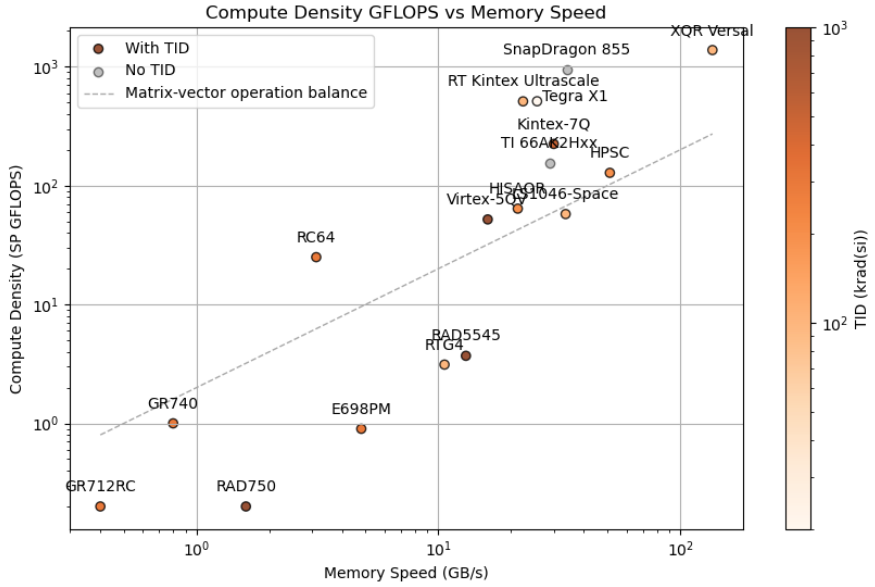


Figure 1. Processor performance over time for heterogeneous and CPU processors. Radiation tolerance is plotted with the color bar. Reproduced from Belsten.[1]

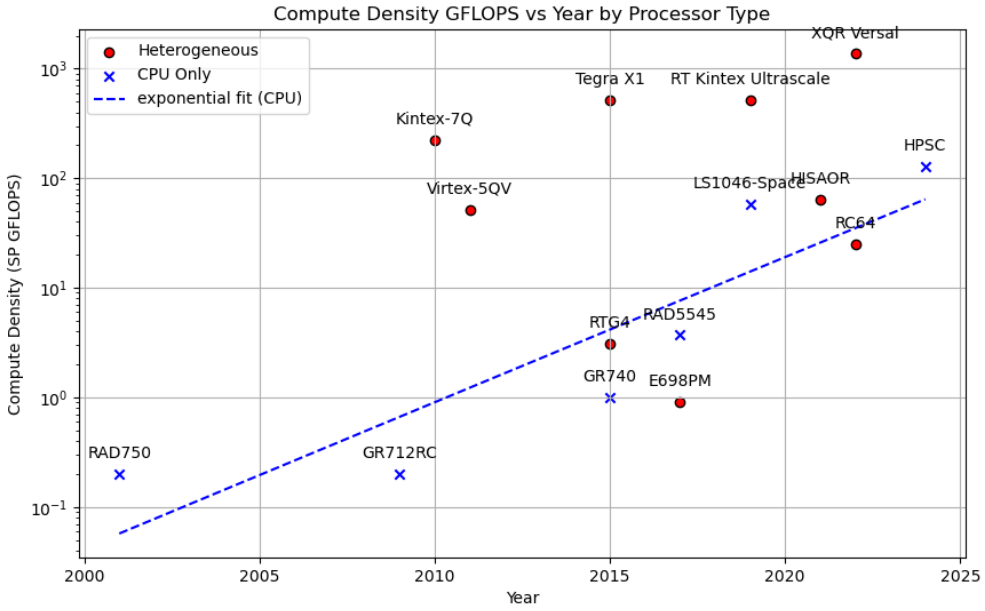


Figure 2. Performance trends compared for heterogeneous and CPU processors. In contrast with Figure 1, here we explicitly identify each processor's architecture type. The typical Moore's Law trend is clearly visible for the CPU processors, and the trend line is indicated. The Moore's Law trend is less clear for heterogeneous processors due to the widely varying architectures. We also see that heterogeneous space processors have become more common in the last decade. Reproduced from Belsten.[1]

To test the improvements possible with characteristic space-relevant FPGAs, we have chosen to evaluate the RT Kintex Ultrascale—at the time of starting our implementation efforts, the higher-performing Versal family of processors was difficult to acquire. The AMD ZCU104 evaluation board, featuring a Kintex Ultrascale+ FPGA, was therefore selected for implementation testing. This platform balances high memory bandwidth, dense DSP resources, and mature development tools while offering a practical stand-in for the RT Kintex Ultrascale flight device. Moreover, the board was readily available for testing, avoiding the access and timeline limitations associated with other top-performing devices like the HPSC and XQR Versal. Previous work has discussed the theoretical improvements possible with FPGA implementations,[1] and the rest of this work is dedicated to demonstrating that FPGAs like the ZCU104 also deliver improved real-world performance, particularly as problem sizes grow to flight-relevant scales.

## 2 HOWFSC CONTROL ALGORITHMS

The HOWFSC system enables high-precision wavefront control for space telescopes, supporting tasks such as speckle suppression and dark hole maintenance. This functionality is driven by two computationally intensive control algorithms: (1) optical modeling, which characterizes the system's sensitivity to actuator perturbations via a Jacobian matrix; and (2) Electric Field Conjugation (EFC), which iteratively computes control commands to suppress unwanted light. These algorithms place significant computational demands on the system and define requirements for acceleration hardware. Specifically, they call for high-throughput, low-latency computational kernels. In this section, we briefly describe each algorithm's role in the system and identify the computational kernels that emerge from their implementation. These kernels form the basis for the design of dedicated accelerators and system-level scheduling.

### 2.1 Optical Modeling

Optical modeling simulates light propagation through the telescope's optics, capturing the influence of deformable mirror (DM) actuation on the science image plane's electric field. It underpins the estimation of the Jacobian matrix, which links DM actuator inputs to optical outputs. The modeling uses Fourier optics, involving propagation steps performed via:

- **2D Fast Fourier Transform (FFT)**
- **Matrix Fourier Transform (MFT)**: two complex matrix multiplications
- **Angular Spectrum Method**: two 2D FFTs + element-wise complex multiplication

Figure 3 shows the full operation scheduling of our model and their dependencies[1]

### 2.2 Electric Field Conjugation (EFC)

Given a Jacobian matrix, EFC computes control updates that minimize the image-plane electric field in the dark hole region. It solves a regularized least-squares problem:

$$\left(J^T J + \alpha^2 10^\beta I\right) \delta_{DM} = -J^T E \tag{1}$$

Typically, via QR decomposition. This leads to the following system-level computational demands:

- **Matrix-matrix multiplication** ($J^T J$) when the Jacobian is updated
- **QR decomposition** when the regularization parameter $\alpha$ is modified
- **Matrix-vector multiplication** for each EFC iteration

Figure 4 shows the full operation scheduling of our model and their dependencies[1]

Table 1. Summary of acceleration kernels and their usage in HOWFSC control bottleneck algorithms.

| **Operation Kernel** | **Algorithm Usage** |
|---|---|
| 2D FFT | Jacobian Estimation, Angular Spectrum Method |
| Angular Spectrum Method | Jacobian Estimation |
| Matrix-Matrix Multiplication | Jacobian Estimation, EFC, MFT |
| MFT | Jacobian Estimation |
| Matrix-Vector Multiplication | Jacobian Estimation |
| QR Decomposition | EFC |

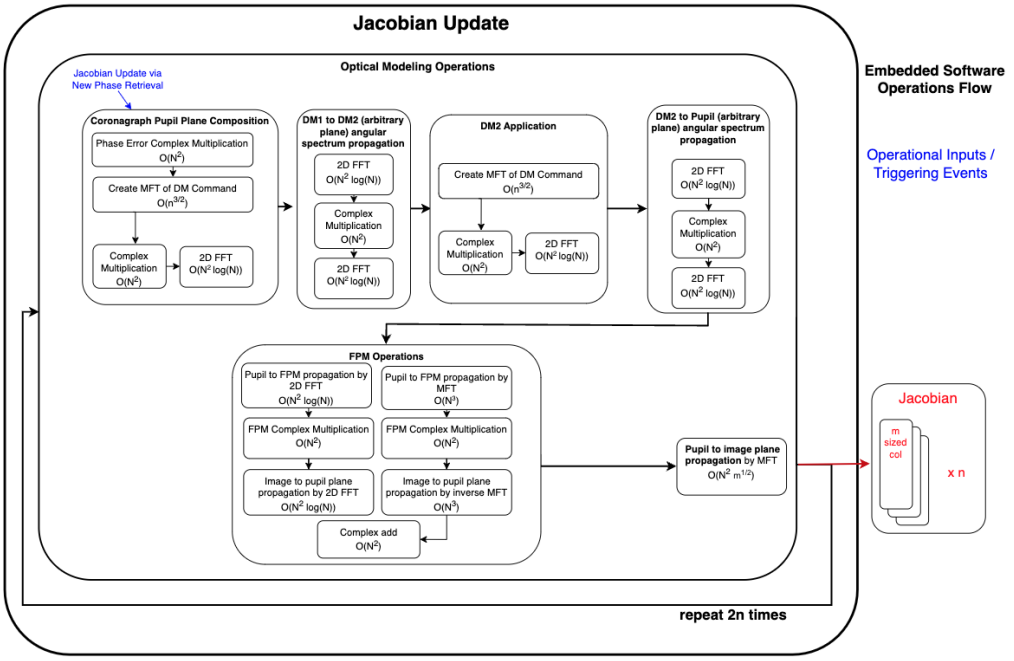


Figure 3. Operations during Jacobian estimation through optical modeling. These include forward and inverse Fourier transforms, transfer function multiplications, and light propagation steps based on telescope structure. Reproduced from Belsten.[1]

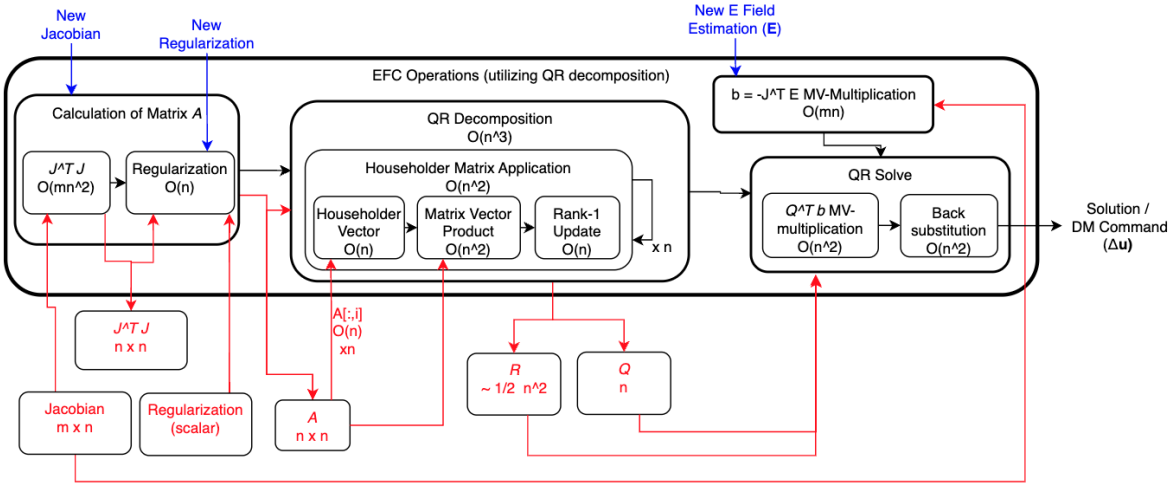


Figure 4. Overview of operations and data storage in EFC based on QR decomposition for solving a linear system of equations. The Jacobian sensitivity matrix, $J$, has dimensions $m \times n$ with $m > n$. The matrix $A$ represents the regularized form of $J^T J$. The decomposition yields matrices $Q$ and $R$, where $Q$ is stored in its factored form. MV-multiplication denotes matrix-vector multiplication. Reproduced from Belsten.[1]

## 3 ACCELERATORS DESIGN

This section describes the architecture and implementation of our custom hardware accelerators targeting key computational bottlenecks in HOWFSC. Each kernel is designed to exploit the inherent parallelism of the target operation while minimizing memory bottlenecks through efficient pipelining, streaming interfaces, and optimized data-flow scheduling. We focus on the Angular Spectrum Method, Matrix Fourier Transform (MFT), and key linear algebra operations used in Electric Field Conjugation (EFC), detailing their structure, performance considerations, and FPGA-specific optimizations.

### 3.1 Angular Spectrum Method

The kernel consists of three major execution blocks—F1, F2, and F3—that execute in strict sequence (see Figure 5). Each block operates internally using a dataflow model, leveraging instruction-level parallelism and optimized scheduling. This sequential execution is required because all three blocks share access to the same memory buffers. As a result, pipelining or overlapping their execution would lead to memory port contention. An alternative design is to allocate three additional buffers, each the size of the input matrix, to hold intermediate outputs, allowing pipelined execution across the major blocks. However, this approach is infeasible for large matrices, as in the case in our application.

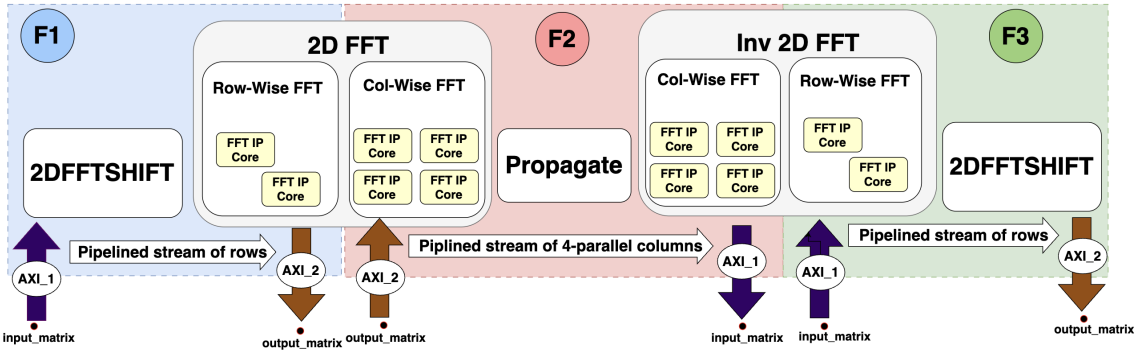


Figure 5. Data flow in the angular spectrum kernel.

Each of the F1, F2, and F3 stages performs a specific part of the 2D Fourier optics pipeline and is structured around the memory streaming layout of the input data. As illustrated in Figure 5, F1 operates on row-major data, performing a row-wise FFT directly as the data is streamed from memory. F2 processes column-major data, enabling column-wise FFTs, pointwise propagation through the medium, and a second column-wise FFT—all without intermediate memory writes. Finally, F3 reverts to a row-wise layout to perform the last FFT step before streaming the result back to memory. This division ensures that each stage maximally exploits burst memory access and minimizes costly data reordering, enabling a fully pipelined and streaming implementation of the angular spectrum method. A hardware architecture block diagram of the complete kernel is provided in Figure 10.

### 3.1.1 F1 Stage

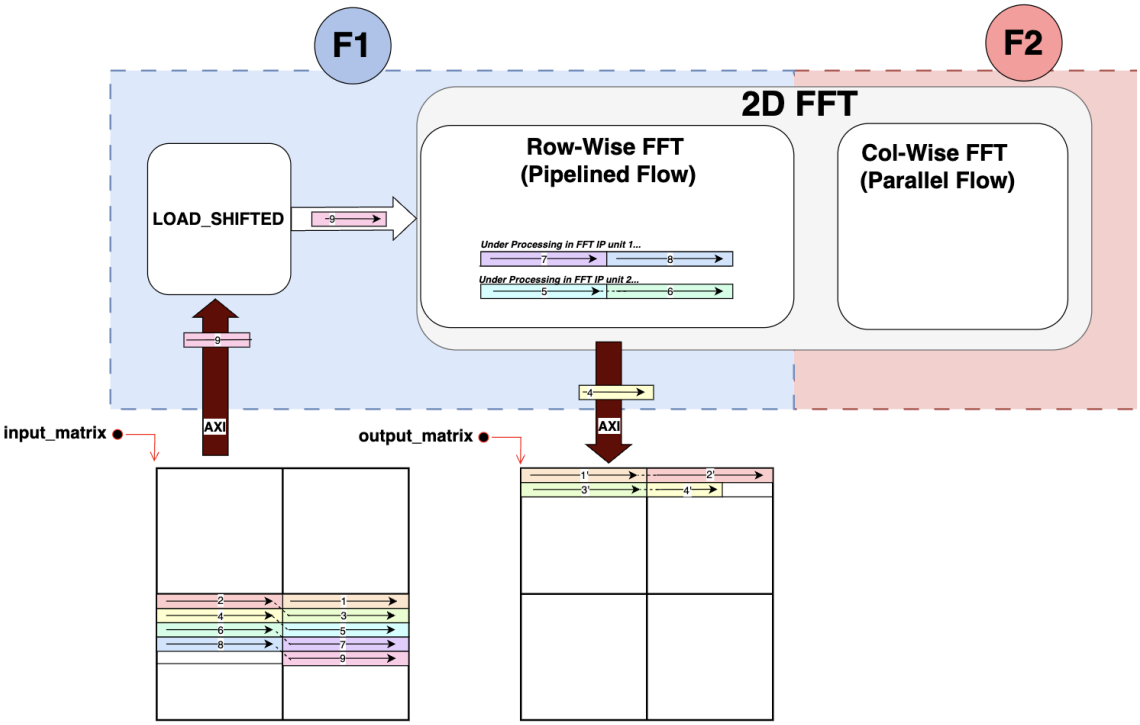


Figure 6. Visualization of Data Flow within block F1. The execution of this stage occurs in a pipelined scheme: while some rows are in the row-wise FFT module, others are being streamed from memory and prepared to enter, while others are being streamed out after computation. A detailed description of this design is provided in Section 3.1.1.

The first block, F1, corresponds to streaming the input wavefront array from memory to the row-wise FFT block of the first 2D FFT (see Figure 6). The algorithm requires performing a 2D FFT shift before the first 2D FFT. However, in our design, we stream the data into the FFT core as if it were already shifted. This functionality is implemented in the module `shifted_stream_to_first_fft`, which streams the data from memory as if the shift operation had already been applied. The resulting output is then streamed back to memory in the shifted format, as all subsequent operations can be performed in this format without reversing the shift. As the data is streamed in row-major order into the row-wise FFT, we take advantage of maximum burst transfers by streaming 8 consecutive complex numbers at a time. **This streaming mechanism is a key part of the performance**.

Two IP cores are used inside the row-wise FFT module to operate in a pipelined scheme (see Figure 7). The `shifted_stream_to_first_fft` module continuously streams rows inside. Once a row arrives, it is sent to the first core for processing. When a second row arrives, it is sent to the second core for processing. This pipelined scheme continues until all rows in the input array are processed.

The optimal number of FFT cores is three, which balances computation throughput with memory bandwidth. Streaming one row from memory takes approximately one-third of the latency of computing its FFT on a single core. Therefore, using three FFT cores allows continuous pipelined execution without stalls, effectively hiding memory latency and avoiding port contention, according to Little's Law.[3] However, this design uses 2 cores for simplicity, as the number of rows is a power of 2. Additionally, using 4 cores would require more resources, given the relatively high utilization per core—and the fourth core would remain idle. This is because the memory bandwidth is insufficient to supply data fast enough to keep all four cores fully utilized, resulting in under-utilization rather than improved performance.

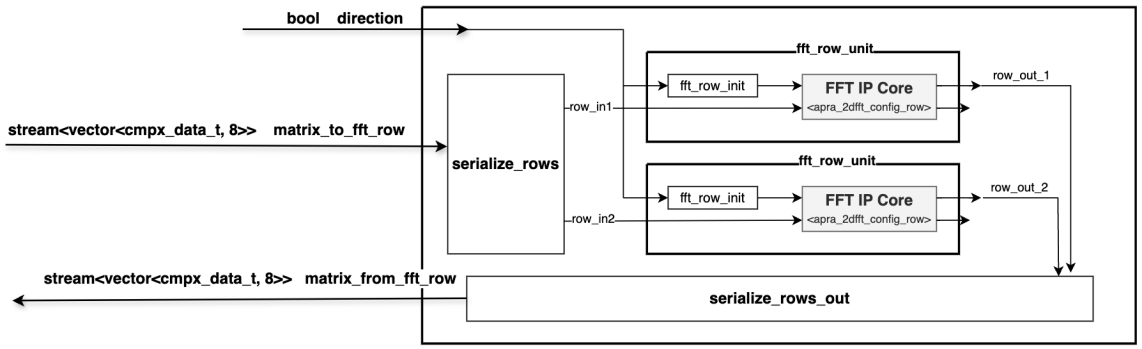


Figure 7. Block diagram of Row-Wise FFT module: rows are serialized to cores as soon as they arrive and serialized out as soon as they are processed. Two cores are used.

### 3.1.2 F2 Stage

The second stage starts with streaming the results of stage 1 (input wavefront array shifted and row-wise FFT applied), streamed into the column-wise FFT.

The `stream_to_col_fft` module streams the data from memory in column-major order, processing 4 columns at a time. Each streamed element is a `vector<complex<data_t>>` containing 4 elements from 4 consecutive columns of row $k$. Once this vector is transfered from memory, the next element contains data from the same 4 consecutive columns of row $k+1$, and so on, until all 4 columns are streamed. Then, the next set of 4 columns is streamed (see Figure 8).

The `fft_col` module contains 4 IP cores that process the input vectors in parallel. See Figure 9. Doubling the number of cores (up to 8) is possible, as the maximum burst transfer can move 8 elements. This would reduce latency at the cost of additional resources. Once the first column-wise FFT is complete, the output is not streamed back to memory but directly passed to the `propagate` module as shown in figure 8.

The `propagate` module multiplies the input data by the corresponding phase transfer function, $e^{ik_z d}$. The propagation matrix is constructed on the fly for each 4-element vector streamed from the column-wise FFT, taking the shifting into consideration.

The next step in the angular spectrum algorithm is another 2D FFT. Since the data is already in column-major order, the second column-wise FFT is performed before the row-wise FFT (equivalent reordering). The output from the `propagate` module is streamed directly to the second column-wise FFT, and the results are streamed back to memory.

Although stage F2 performs the majority of the computation (including 2 column-wise FFTs and propagation), it accounts for only 1/5 of the total execution time due to the reduction in data movement and efficient parallelism.

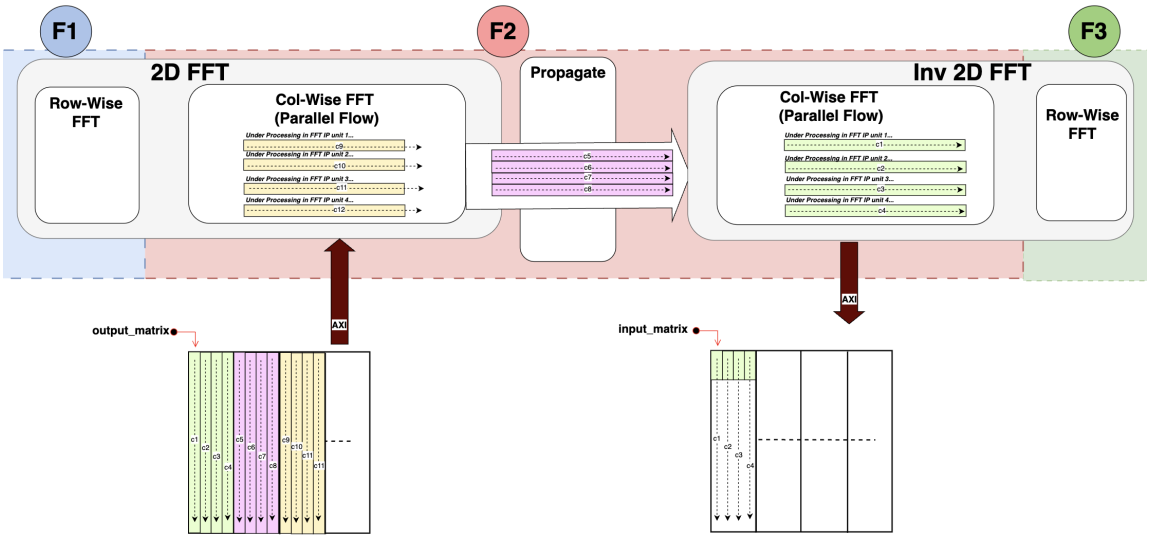


Figure 8. Execution in stage 2 follows a dataflow scheme. For example, while columns $k, k+1, k+2, k+3$ are being streamed into the first column-wise FFT, columns $k+4, k+5, k+6, k+7$ are being processed in the `propagate` module, and columns $k+8, k+9, k+10, k+11$ are processed in the second column-wise FFT. This design minimizes data movement and avoids additional memory accesses for reordering.

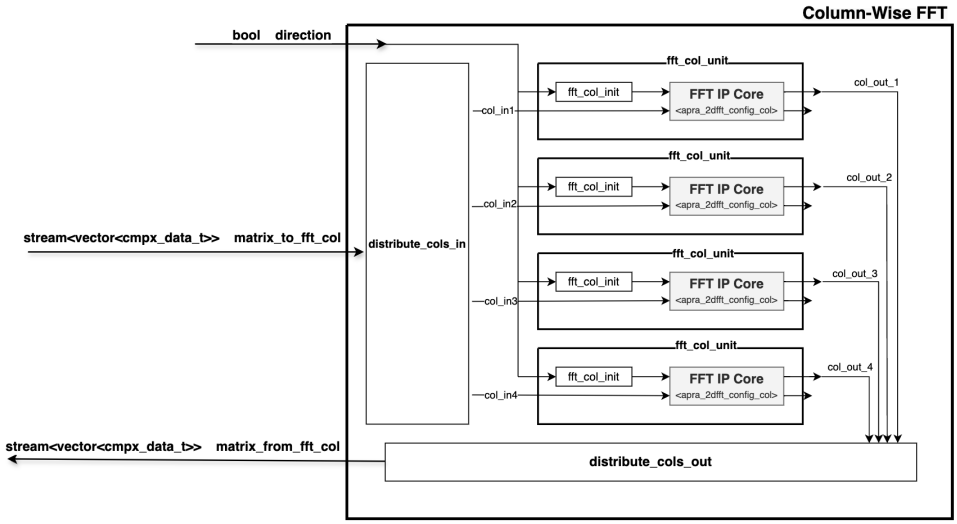


Figure 9. Block diagram of Column-Wise FFT module: columns are streamed 4 at a time, distributed over 4 IP cores, processed in parallel, and streamed out.

Following the propagation and second column-wise FFT in Stage F2, the data must be transformed back to row-major format to complete the angular spectrum method. Stage F3 performs this final step.

### 3.1.3 F3 Stage

In this stage, a row-wise FFT is applied to the column-wise output, mirroring the process used in F1. This transforms the data back into the spatial domain and prepares it for post-processing or storage. The output is streamed back to memory in an inverse-shifted format, ensuring that the entire angular spectrum computation is completed without requiring explicit reordering or buffering between stages.

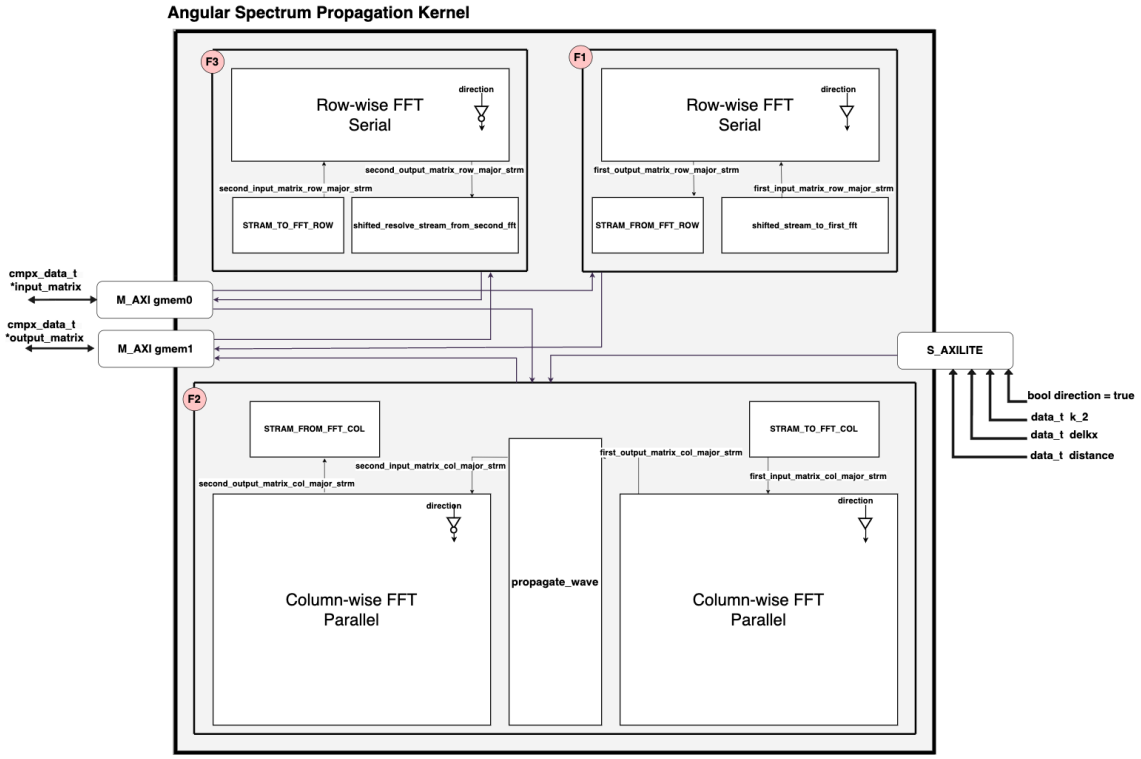


Figure 10. Architectural block diagram of Angular Spectrum Method kernel

### 3.2 Matrix Fourier Transform (MFT)

The *Matrix Fourier Transform* treats the two-dimensional discrete Fourier transform as two dense matrix products. Given an $M \times N$ signal matrix $A$, we pre-compute the one-dimensional DFT matrices $F_M \in \mathbb{C}^{M\times M}$ and $F_N \in \mathbb{C}^{N\times N}$ with entries $\left[F_M\right]_{u,v} = e^{-j2\pi uv/M}$ and $\left[F_N\right]_{p,q} = e^{-j2\pi pq/N}$. The 2-D transform is then obtained by

$$\mathrm{MFT}(A) = F_M \, A \, F_N^{\mathrm{H}},$$

where $(\cdot)^{\mathrm{H}}$ denotes the Hermitian (conjugate-transpose). Thus, the numerical core of every MFT is a pair of complex general matrix multiplications (CGEMM) operations: first a left multiplication by $F_M$ and then a right multiplication by $F_N^{\mathrm{H}}$.

Our implementation invokes the CGEMM kernel twice: first, the signal matrix is multiplied by $F_M$ and then by $F_N^{\mathrm{H}}$. Figure 11 in the appendix depicts the data-flow for a single CGEMM invocation; the same structure is reused for both passes. The kernel is decomposed into three dataflow stages, **F1**–**F3**. Although each stage streams data at one vector per cycle, they are executed strictly in sequence to prevent contention on the shared dual-port BRAMs `ram_A`, `ram_B`, and `ram_C`.

#### 3.2.1 F1 - Load Stage

F1 scans external memory, fetching one $TILE\_SIZE \times TILE\_SIZE$ tile from each of the three operand matrices(`A`, `B`, `C`). The loader issues 512-bit AXI bursts that carry $VECTOR\_SIZE = 4$ (for a complex number made up of doubles, 8 for floats) complex numbers per beat, achieving full bus utilization. Each burst is grouped into a `stream<vector_t>` and pushed into the dataflow region with fifos and interconnects handled automatically by the HLS runtime. Consequently, four complex values for every matrix arrive on the chip every cycle, enabling the compute stage to run without stalling.

#### 3.2.2 F2 - Computation

On entry, F2 performs a "preload" that drains the three input streams into the on-chip buffers (`ram_A`, `ram_B`, `ram_C`), thereby decoupling the subsequent arithmetic from memory latency. Computation then proceeds through the nested loops `cmmult_outer` (rows $i$), `cmmult_mid` (vectorized columns $j$), and `cmmult_inner` (scalar products $k$), exactly as annotated in Figure 11. The outer $i$ and $j$ loops are pipelined with an initiation interval of1̃6, while the inner $k$ loop is fully unrolled; hence $TILE\_SIZE \times TILE\_SIZE$ complex multiply–accumulate operations are launched toward the DSP array *every* cycle. Partial products are buffered in a register file (`temp_arr`), reduced to a vector-wide sum, and finally accumulated element-wise into `ram_C`, keeping all arithmetic local to FPGA resources and eliminating extra memory traffic.

#### 3.2.3 F3 - Store Result/Writeback

When the tile update completes, F3 streams the modified `ram_C` contents back to DRAM via `store_result`. Like the loader, writes are vector-burst aligned so that four complex numbers depart the device on every clock edge. Because `ram_C` is read-only for F1 and F2, and write-only for F3, the sequential schedule completely removes BRAM port hazards. The loop nest of Figure 11 iterates over all $\lceil M/TILE\_SIZE \rceil \times \lceil P/TILE\_SIZE \rceil$ tiles, after which the host launches the second CGEMM pass to finish the two-sided MFT.

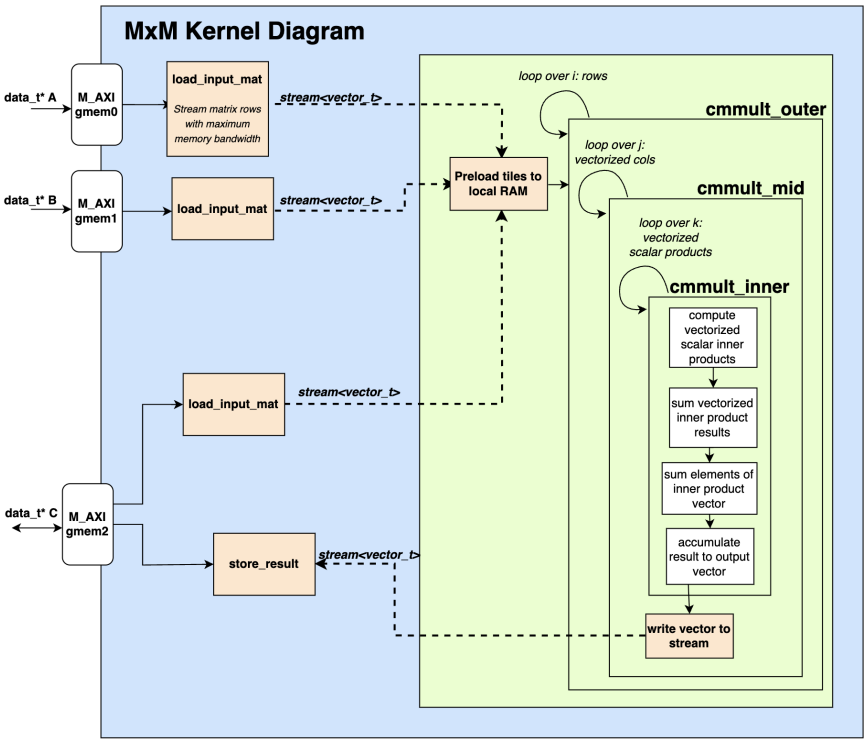


Figure 11. Block diagram of Complex Matrix Product kernel

### 3.3 EFC: QR Givens and Matrix-Vector Multiplication

The numerical implementation of Electric Field Conjugation (EFC) in this work baselines the explicit formation of the Gram matrix $J^T J$, followed by a QR decomposition to solve the resulting regularized linear system.[1] This choice is informed by the need for a stable and general-purpose approach that accommodates the numerical characteristics of typical HOWFSC Jacobians. We also consider the case of a precomputed pseudo-inverse matrix. In this case, the only necessary numerical operation is the matrix-vector product.

To determine the full regularized linear least-squares solution, alternative methods such as LU and Cholesky decompositions were also evaluated. QR decomposition was ultimately selected for its numerical robustness, particularly when dealing with potentially ill-conditioned systems. Direct SVD-based approaches were also considered, both on the Gram matrix and on the tall Jacobian $J$ itself. However, these were rejected as baselines due to their significantly higher computational cost. In practice, QR decomposition following Gram matrix formation remains faster and sufficiently robust for our application.

When solving the regularized linear least-squares (LLSQ) problem for EFC, the overall computational burden is dominated by two distinct stages: Gram matrix formation and QR decomposition.

The first bottleneck arises from the explicit construction of the Gram matrix $J^T J$, where $J$ is the $m \times n$ Jacobian matrix. This step requires accumulating all pairwise dot products between

the columns of $J$, and its cost scales with both the number of measurements $m$ and the number of control degrees of freedom $n$. If the Jacobian is not modified between iterations, the Gram matrix can be precomputed for multiple iterations. We accelerate the Gram matrix formation with a matrix-matrix product kernel analogous to that developed for the MFT above.

The second bottleneck occurs during the solution of the linear system 1

$$\left(J^T J + \alpha^2 10^{\beta} I\right) \delta_{DM} = -J^T E$$

which requires factoring the $n \times n$ coefficient matrix. QR decomposition is used to obtain a stable solution. QR decomposition of the $n \times n$ Gram matrix requires approximately $\frac{4}{3}n^3$ floating-point operations, making it one of the dominant costs when solving the regularized linear least-squares system.

We considered three overall algorithms for computing the QR decomposition: Gram–Schmidt orthogonalization, Householder reflections, and Givens rotations. Of these options, Givens rotations most lend themselves to our MPSoC implementation because rows of $Q$ and $R$ can be streamed in and operated on with pipelining (and with parallelization across the elements of each row).

To perform the QR decomposition via Givens rotations, we must operate on $\mathcal{O}(N^2)$ rows sequentially due to inter-loop dependence. However, these operations can be pipelined and parallelized internal to each row $\mathcal{O}(N)$, so we achieve an overall speedup of up to $N$. Our implementation is summarized in Figure 12.[1]

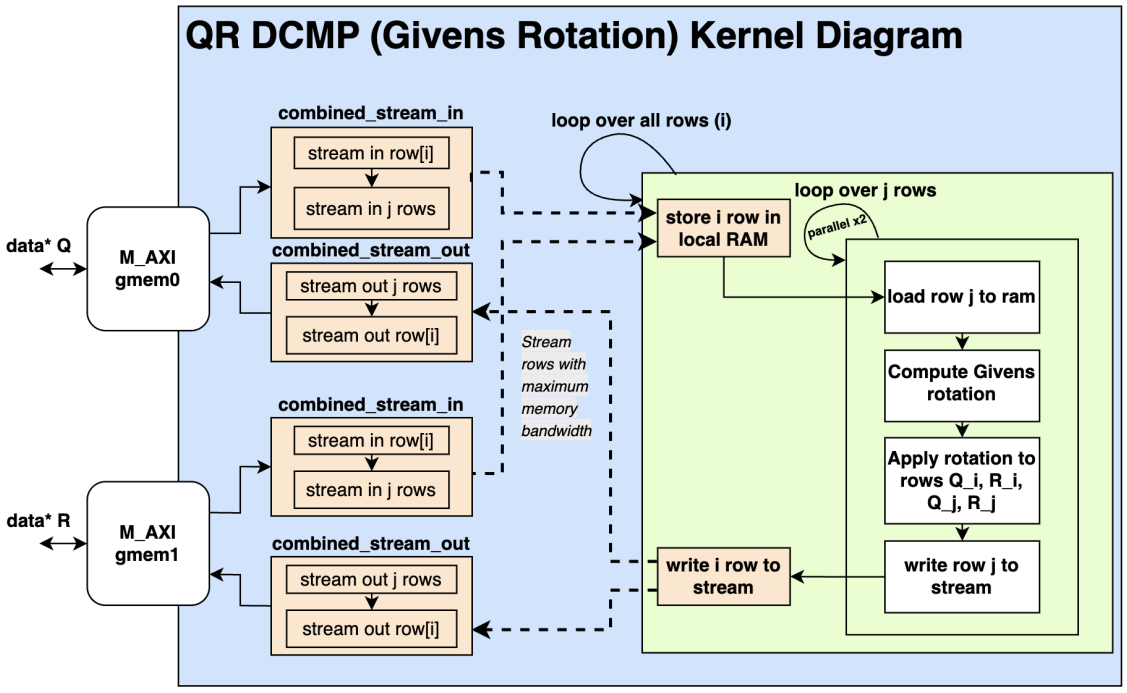


Figure 12. Diagram of the QR decomposition kernel for the UltraScale+ MPSoC. Figure reproduced from.[1]

If the regularization parameter is kept the same between iterations, the QR decomposition can also be precomputed. In this case, we can even explicitly compute the regularized pseudo-inverse of the Jacobian matrix, known as the Command Matrix. This explicitly relates an input electric field state to the desired DM actuator command. This matrix is of the same size as the Jacobian but with transposed dimensions.

With all precomputations in place, solving EFC at each iteration consists only of a single matrix-vector product. Accelerating the matrix-vector product on the FPGA is therefore desired.

The challenge in the matrix-vector product lies in loading the matrix from memory. The primary design constraint is to efficiently load the matrix from memory and perform the inner product without slowing down the matrix loading. We handle this by pre-loading the input vector into fabric RAM and streaming the matrix contents into a pipelined compute loop, which multiplies the matrix element with the correct vector element and accumulates the result. To maximize the available data interface, we stream in HLS vectors containing a number of elements such that the vector bit-width matches the AXI memory interface (512 bits). We utilize an adder tree to efficiently accumulate the entire inner product onto each element in the result vector. An entire vector's worth of rows is operated on in the pipeline so that a full HLS vector-sized chunk of the output vector can be efficiently streamed out.[1] This design is summarized in Figure 13.

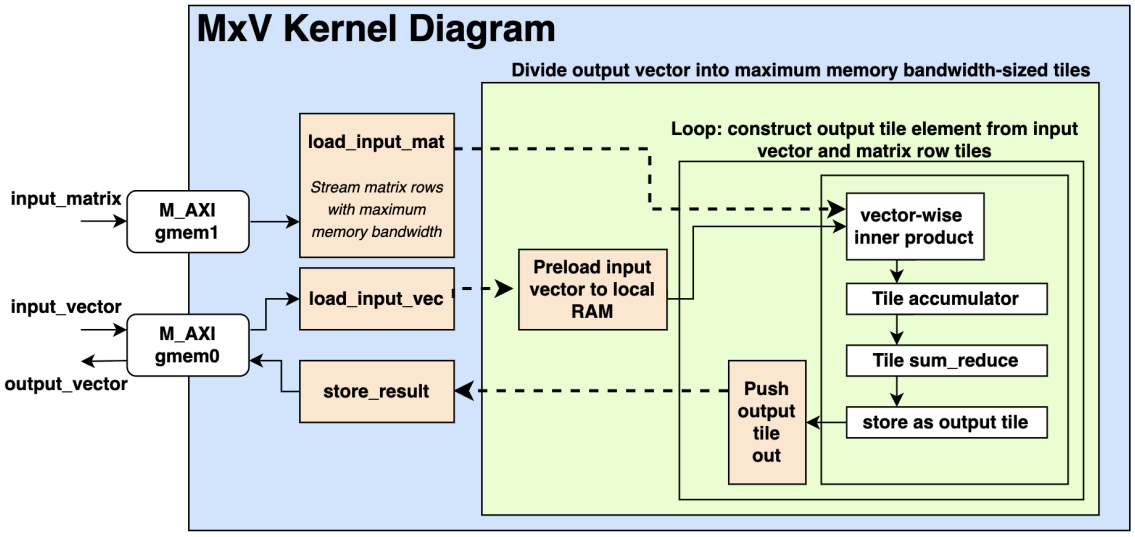


Figure 13. Diagram of the matrix-vector product kernel for the UltraScale+ MPSoC. Figure reproduced from.[1]

# 4 RESULTS

This section presents real-world execution timings for our FPGA kernel, compared against the baseline application processor on the MPSoC ZCU104 board (Arm Cortex-A53).

## 4.1 Angular Spectrum Method and 2D FFT Performance

We evaluate both the 2D FFT and the full Angular Spectrum Method. For additional context, we also include performance results from high-performance processors such as the Apple M1. Software implementations used include Numerical Recipes in C[4] and Python's NumPy library, which leverages SIMD acceleration. Due to limitations of the Xilinx FFT IP core—which supports only fixed-point and single-precision floating-point (but not double-precision)—all hardware timing results are based on single-precision floating-point computation.

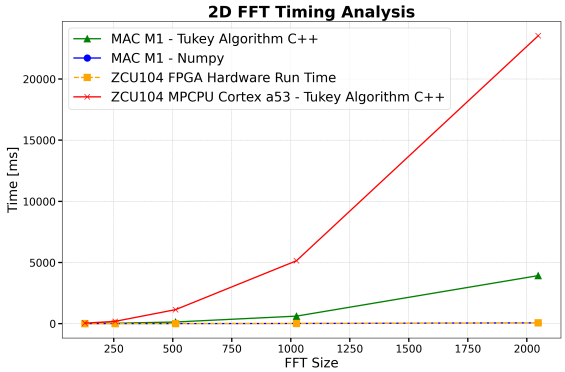


Figure 14. Execution time of 2D FFT on different platforms (linear scale).

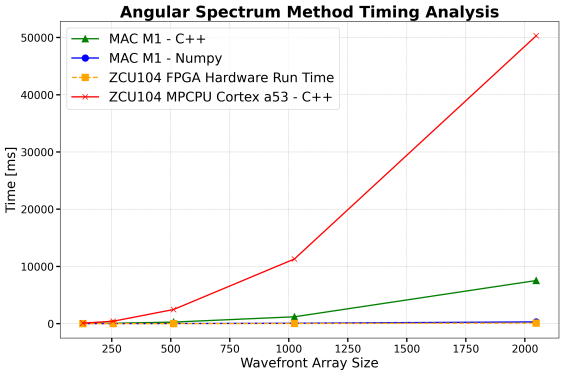


Figure 15. Execution time of Angular Spectrum Method on different platforms (linear scale).

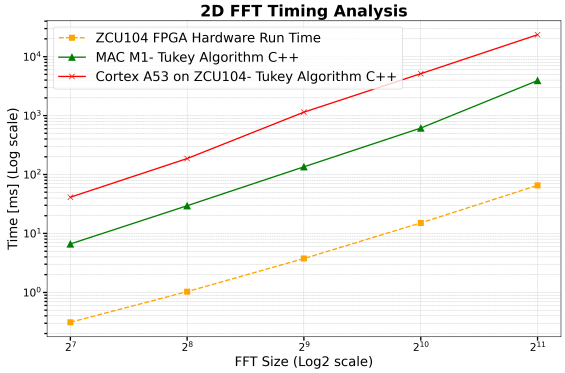


Figure 16. Log-log scale of 2D FFT FPGA execution time. Linear fit to FPGA times: $\log(\text{time}) = 1.338 \cdot \log_2(n) - 17.536$.

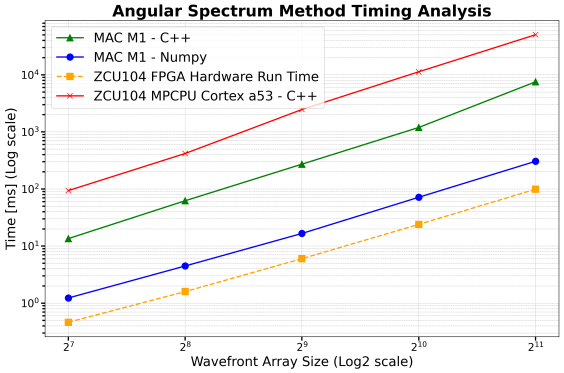


Figure 17. Log-log scale of Angular Spectrum Method FPGA execution time. Linear fit to FPGA times: $\log(\text{time}) = 1.344 \cdot \log_2(n) - 10.243$.

Table 2 summarizes the execution times and speedup factors of the FPGA implementation of the Angular Spectrum Method compared to various software baselines. The FPGA implementation

achieves up to **500× speedup** over the Cortex-A53 processor, **75× over Apple M1 C++**, and **3× over Apple M1 NumPy** for input matrix size **n = 2048**.

Table 2. Acceleration Factors for Angular Spectrum Method

| **Size** | **FPGA** | **ARM Cortex-A53** | | **Apple M1 C++** | | **Apple M1 NumPy** | |
|---|---|---|---|---|---|---|---|
| **n** | **Seconds** | **Seconds** | **Speedup** | **Seconds** | **Speedup** | **Seconds** | **Speedup** |
| 128 | 0.000464 | 0.093813 | 202.2 | 0.0136593 | 29.4 | 0.001685 | 3.6 |
| 256 | 0.001596 | 0.418738 | 262.4 | 0.0632588 | 39.6 | 0.004513 | 2.8 |
| 512 | 0.006024 | 2.468463 | 409.8 | 0.273278 | 45.4 | 0.016742 | 2.8 |
| 1024 | 0.023984 | 11.302552 | 471.3 | 1.1969 | 49.9 | 0.071771 | 3.0 |
| 2048 | 0.099250 | 50.312646 | 506.9 | 7.53648 | 75.9 | 0.305984 | 3.1 |

The following table presents performance results for the 2D FFT implementation. Notably, the NumPy implementation performs particularly well for small matrix sizes, as more data lines fit into the Apple M1 cache.

Table 3. Acceleration Factors for 2D FFT

| **Size** | **FPGA** | **ARM Cortex-A53** | | **Apple M1 C++** | | **Apple M1 NumPy** | |
|---|---|---|---|---|---|---|---|
| **n** | **Seconds** | **Seconds** | **Speedup** | **Seconds** | **Speedup** | **Seconds** | **Speedup** |
| 128 | 0.000311 | 0.041216 | 132.5 | 0.0066333 | 21.3 | 0.000156 | 0.5 |
| 256 | 0.001030 | 0.186729 | 181.3 | 0.0295523 | 28.7 | 0.000562 | 0.6 |
| 512 | 0.003753 | 1.144043 | 304.8 | 0.135167 | 36.0 | 0.002917 | 0.8 |
| 1024 | 0.015089 | 5.142044 | 340.8 | 0.614293 | 40.7 | 0.013694 | 0.9 |
| 2048 | 0.065435 | 23.538236 | 359.7 | 3.92586 | 60.0 | 0.065932 | 1.0 |

## 4.2 QR—Givens

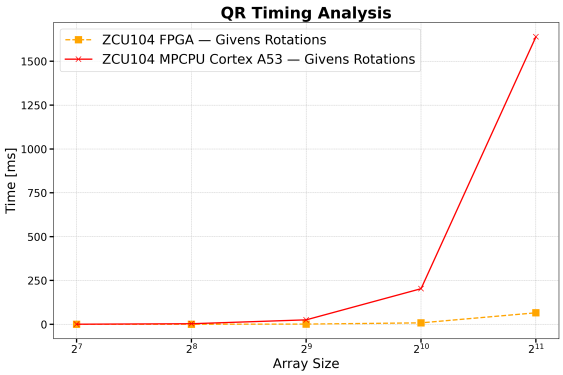


Figure 18. Linear scale of QR execution time.

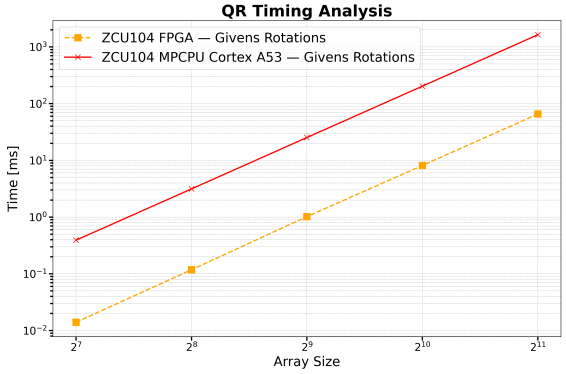


Figure 19. Log-log scale of QR execution time. Linear fit: $\log(\text{time}) = 1.344 \cdot \log_2(n) - 10.243$.

The following table presents performance results for the QR-Givens implementation:

Table 4. Acceleration Factors for QR (Givens Rotations algorithm, float)

| Size | FPGA | ARM Cortex-A53 | |
|---|---|---|---|
| n | Seconds | Seconds | Speedup |
| 128 | 0.014 | 0.392 | 28.0 |
| 256 | 0.118 | 3.140 | 26.6 |
| 512 | 1.020 | 25.300 | 24.8 |
| 1024 | 8.130 | 203.000 | 25.0 |
| 2048 | 65.500 | 1640.000 | 25.0 |

## 4.3 MFT

Here we evaluate both a single complex matrix product (numerical core of the MFT) and the full MFT computation. Results are shown for both double- and single-precision floating-point numbers. It is worth noting that more optimized kernels can be constructed for this computation.

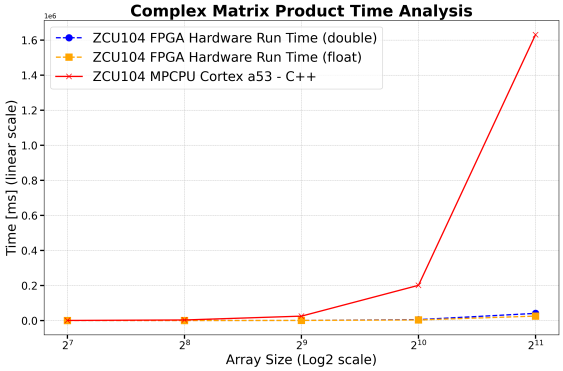


Figure 20. Linear scale of CGEMM execution time on CPU and FPGA.

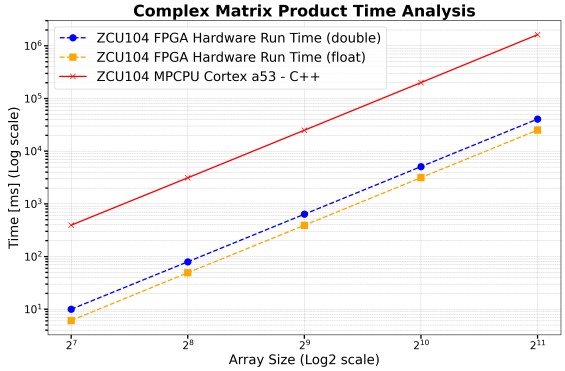


Figure 21. Log scale of CGEMM execution time on CPU and FPGA.

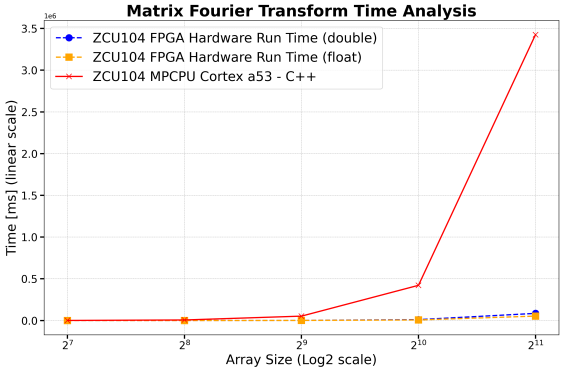


Figure 22. Linear scale of MFT execution time on CPU and FPGA.

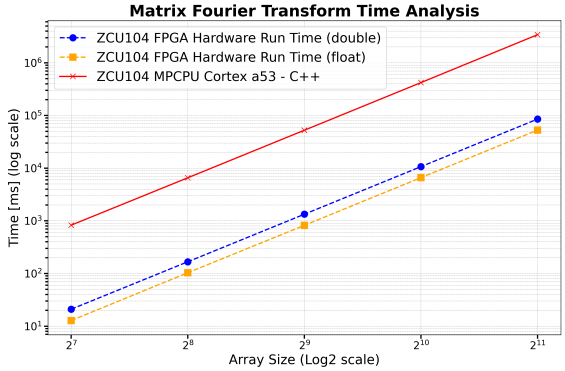


Figure 23. Log scale of MFT execution time on CPU and FPGA.

Table 5 summarizes execution times and speedup factors of the FPGA implementation of Complex GEMM (CGEMM) compared to the software baseline, for both single- and double-precision

numbers. The FPGA implementation achieves up to **64.8× speedup** over the Cortex-A53 processor for matrix size **n = 2048**.

Table 5. Acceleration Factors for CGEMM

| **Size** | **ARM Cortex-A53** | **FPGA (double)** | | **FPGA (float)** | |
|---|---|---|---|---|---|
| **n** | **Seconds** | **Seconds** | **Speedup** | **Seconds** | **Speedup** |
| 128 | 0.3948 | 0.0100 | 39.6 | 0.0061 | 64.9 |
| 256 | 3.1300 | 0.0796 | 39.3 | 0.0429 | 63.5 |
| 512 | 25.032 | 0.6362 | 39.3 | 0.3916 | 63.9 |
| 1024 | 200.84 | 5.0944 | 39.4 | 3.1496 | 63.8 |
| 2048 | 1631.2 | 40.732 | 40.1 | 25.190 | 64.8 |

The following table shows performance results for the full MFT computation:

Table 6. Acceleration Factors for MFT

| **Size** | **ARM Cortex-A53** | **FPGA (double)** | | **FPGA (float)** | |
|---|---|---|---|---|---|
| **n** | **Seconds** | **Seconds** | **Speedup** | **Seconds** | **Speedup** |
| 128 | 0.8093 | 0.0209 | 38.7 | 0.0128 | 63.4 |
| 256 | 6.4166 | 0.1672 | 38.4 | 0.1035 | 62.0 |
| 512 | 51.316 | 1.3361 | 38.4 | 0.8223 | 62.4 |
| 1024 | 411.71 | 10.698 | 38.5 | 6.6142 | 62.3 |
| 2048 | 3343.9 | 85.538 | 39.1 | 52.898 | 63.2 |

# 5 SUMMARY AND FUTURE WORK

## 5.1 Summary

We demonstrated the successful FPGA acceleration of key bottleneck algorithms in high-order wavefront sensing and control (HOWFSC), achieving significant speedups compared to a baseline ARM Cortex-A53 CPU implementation. Table 7 summarizes the performance improvements for input size $n = 2048$ using single-precision floats.

The most substantial gains were observed in operations with high compute-to-memory ratios and streaming-friendly dataflows, such as the Angular Spectrum Method (507×) and 2D FFT (360×). Matrix-matrix and matrix-vector operations also benefited from acceleration, albeit to a lesser extent, due to memory bandwidth limitations, as detailed in the discussion section 5.2. The decomposition of QR using Givens rotations achieved a 25× speedup, confirming the feasibility of stable linear system solutions on the FPGA fabric. These results validate the effectiveness of heterogeneous computing platforms, such as the AMD UltraScale+ MPSoC, in enabling low-latency HOWFSC onboard future space telescopes.

Table 7. Acceleration Factors Summary. Working set $n = 2048$. ZCU104 Platform: CPU vs. FPGA

| **Operation Kernel** | **FPGA Speedup** |
|---|---|
| Angular Spectrum Method | 506.9× |
| 2D FFT | 359.7× |
| Matrix-Matrix Multiplication | 64.8× |
| Matrix Fourier Transform (MFT) | 63.2× |
| QR Decomposition | 25.0× |

### 5.2 Discussion

All methods demonstrated faster execution when accelerated on the FPGA compared to the MPSoC Application Processing Unit (ARM Cortex-A53 CPU), highlighting the advantages of heterogeneous computing with FPGA acceleration. A more comprehensive comparison against dedicated CPU-based systems is beyond the scope of this work and is addressed in related studies such as by Belsten.[1] These findings support the heuristic introduced earlier: FPGAs can significantly outperform CPUs, albeit at the cost of increased development complexity.

However, the observed speedup was highly non-uniform across different numerical methods and varied significantly with the working set size and processor architectural features such as cache size and operating frequency. For instance, the 2D FFT exhibited a speedup of approximately 360× on the FPGA, whereas the matrix-matrix product yielded only a modest improvement of 65×. The Angular Spectrum kernel, which builds upon the 2D FFT, benefited from a dataflow structure that allowed streaming computation, leading to more effective acceleration. Larger working sets also resulted in better performance for FFT-based methods, as the fixed cache size of CPUs became a limiting factor.

One general challenge faced in our FPGA implementations was the bottleneck introduced by external memory bandwidth, especially for dense linear algebra routines such as matrix-vector multiplication. Applications that repeatedly operate on the same data were better suited for FPGA acceleration, as they could more effectively exploit the inherent parallelism without being constrained by memory access delays.

These results suggest a more nuanced understanding of architectural trade-offs. FPGAs (and by extension ASICs) are not universally superior to CPUs by a fixed performance margin. Rather, each architecture offers distinct advantages depending on the computational characteristics of the algorithm. While we observed acceleration across all methods tested, the magnitude of improvement ranged from marginal to multiple orders of magnitude. This underscores the importance of analyzing the computational composition of a workload when selecting an appropriate processing architecture.

### 5.3 Future Work

Based on our insight into computational composition above, it is interesting to consider processors that lend themselves well to dense linear algebra. Such a processor could efficiently perform operations that are bottlenecked by external memory bandwidth, such as the matrix-vector product and QR decomposition. Processors well-suited to these tasks include GPUs and memory-parallel processor architectures. The investigation of these architectures for space-based execution of HOWFSC methods should be a subject of future work.

Another direction for future work is the porting of these algorithms to more modern FPGA and CPU platforms, particularly the XQR Versal and the HPSC. We expect that the relative merits and challenges of the various processor architectures will remain consistent in this newer generation of devices, but higher speeds may be achieved, advancing the state of the art in execution time for these key real-time algorithms. Furthermore, future work will include full utilization of heterogeneous compute resources, such as embedded GPUs and AI Engines available in platforms like Versal. These components may provide additional acceleration opportunities for key kernels and further reduce system-level latency.

## ACKNOWLEDGMENTS

This work is supported by the NASA Astrophysics Technology Division under APRA grant #80NSSC22K1412.

## REFERENCES

[1] Belsten, N., *Embedded Computing for Wavefront Control on Future Space Telescopes*, ph.d. thesis, Massachusetts Institute of Technology, Cambridge, MA (June 2025). Licensed under CC BY-SA 4.0.

[2] Eickert, B., *Accelerating Embedded HOWFSC Algorithms*, masters thesis, Massachusetts Institute of Technology, Cambridge, MA (June 2025). Licensed under CC BY-SA 4.0.

[3] Hennessy, J. L. and Patterson, D. A., [*Computer Architecture: A Quantitative Approach*], Morgan Kaufmann (2011).

[4] Press, W. H., Teukolsky, S. A., Vetterling, W. T., and Flannery, B. P., [*Numerical Recipes in C: The Art of Scientific Computing*], Cambridge University Press, New York, NY, USA, 2nd ed. (1992).

# A Platform Development: Angular Spectrum Propagation Kernel

## A.1 Development Stack

The system is designed to run on Zynq UltraScale+ MPSoCs, which integrate programmable logic (FPGA fabric) with powerful application processing units (CPUs and GPUs). Advanced devices like AMD's Versal series also incorporate AI engines for accelerated computation. Development for such systems involves two primary components: the hardware-accelerated kernel, implemented and flashed onto the programmable logic (PL), and the host software application, which operates on the embedded CPU. The host software communicates with the hardware kernel via the Xilinx Runtime (XRT) library, running on a Linux-based operating system, such as Petalinux.

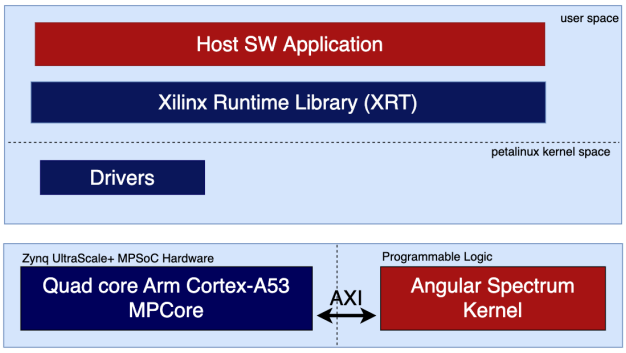


Figure 24. Development stack of the system. The base hardware platform of the Zynq UltraScale+ MPSoC is extended with the configured PL angular spectrum kernel. The host application controls the kernel through XRT, with data movement between the computing unit and CPU/memory managed via the AXI interface.

The hardware design process begins with specifying the functionality of the PL kernel. Developers may opt for traditional Register-Transfer Level (RTL) coding in SystemVerilog or VHDL or utilize High-Level Synthesis (HLS) tools, such as AMD's Vitis, to write designs in high-level languages like C++, then converted by synthesis to an RTL design. In this work, we chose the HLS approach. The AMD Vivado and Vitis tools are used to compile and synthesize the C++ design into a PL kernel (.xo file), which is linked with the base hardware platform using the Vitis v++ linker. This process produces a complete hardware configuration in a Xilinx Support Archive (.xsa) file.

Simultaneously, the host software application is developed to manage system operations. This includes generating or loading input data, controlling the hardware kernel via XRT, and orchestrating data movement between the host CPU and the PL kernel. The final software, along with the hardware container, platform configuration files, and the Petalinux operating system image, is packaged and built into a bootable system image. This package includes the FPGA bitstream, the binary container (.xclbin), and all necessary software components.

At runtime, the system boots with the generated image. The host application loads the binary container file to configure the PL kernel, loads or generates input data, and synchronizes the data with the global DDR memory. It then launches the PL kernel for computation and retrieves the output data upon completion.

## A.2 Software—Hardware Interface of The Kernel

The hardware programmable logic kernel interfaces with memory and register interface paradigms that follow the AXI interface protocol, utilizing an adapter to communicate with external components. The memory paradigm ('m_axi') specifies a memory-mapped interface used by the kernel to access data through DDR. In the register paradigm ('s_axilite'), the data is accessed by the kernel via register interfaces, while software accesses these registers through read/write operations.

The HLS kernel has the following interface:

```
void angular_spectrum(
        bool direction,      // Forward propagation direction
        data_t distance,     // Propagation distance
        data_t k_2,          // Norm square of the wavenumber vector
        data_t delkx,        // 2 * PI / (pixel_scale * n)
        complex<data_t> *input_mat,
        complex<data_t> *output_mat
    );
```

Listing 1. Angular Spectrum Kernel Interface

The kernel is software-controlled through XRT by reading and writing control registers on the s_axilite interface. This interface provides functionality for setting control signals to manage kernel execution and operation, transferring scalar inputs, and configuring m_axi base addresses. The memory-mapped interfaces utilize two separate bundles, each supporting one read and one write channel. This design requires two distinct bundles for the provided array buffers, as both reading and writing operations are necessary during the design to handle intermediate-stage buffers.

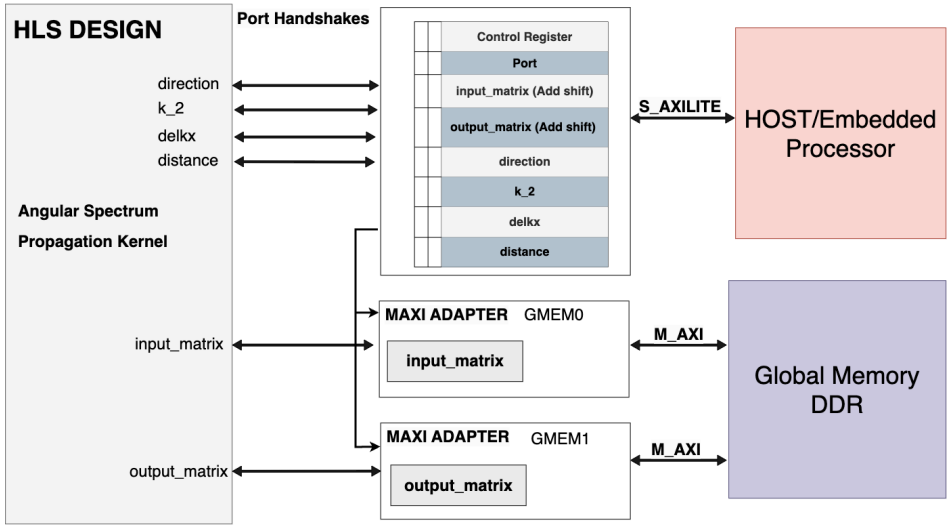


Figure 25. Top-Level Interface of the Angular Spectrum Kernel. Constant scalar arguments (direction, distance, k_2, delkx), as well as base address and control register information, are transferred through s_axilite. Two memory bundles with bidirectional channels are used to interface with DDR for accessing "input_mat" and output_mat.

### A.3 FPGA Resource Utilization

Table 8 summarizes post-placement resource utilization of the Angular Spectrum Method kernel on the Xilinx ZCU104 (UltraScale+ MPSoC), synthesized using Vitis HLS. The design remains within the device limits while leveraging significant DSP and Block RAM resources. This is provided for reference for future developers.

Note that various trade-offs are possible between BRAM and URAM usage, as well as between DSPs and CLB logic. Additionally, latency can be scaled up/down by adjusting the number of FFT IP cores: the current implementation uses 4 cores for column-wise FFTs and 2 for row-wise FFTs, though increasing this up to 8 and 3, respectively, may provide marginal benefits before further scaling becomes ineffective.

Table 8. Angular Spectrum Kernel: Post-Placement Utilization (ZCU104)

| **CLB Logic** | | |
|---|---|---|
| Site Type | Used | Utilization (%) |
| CLB LUTs | 105,965 | 45.99 |
| CLB Registers | 149,957 | 32.54 |
| CARRY8 | 3,221 | 11.18 |
| F7 Muxes | 2,203 | 1.91 |
| F8 Muxes | 989 | 1.72 |
| F9 Muxes | 0 | 0.00 |
| **Memory** | | |
| Block RAM Tiles | 193 | 61.86 |
| URAM | 1 | 1.04 |
| **Arithmetic** | | |
| DSPs | 1,163 | 67.30 |

## B 2D FFT Block Diagram

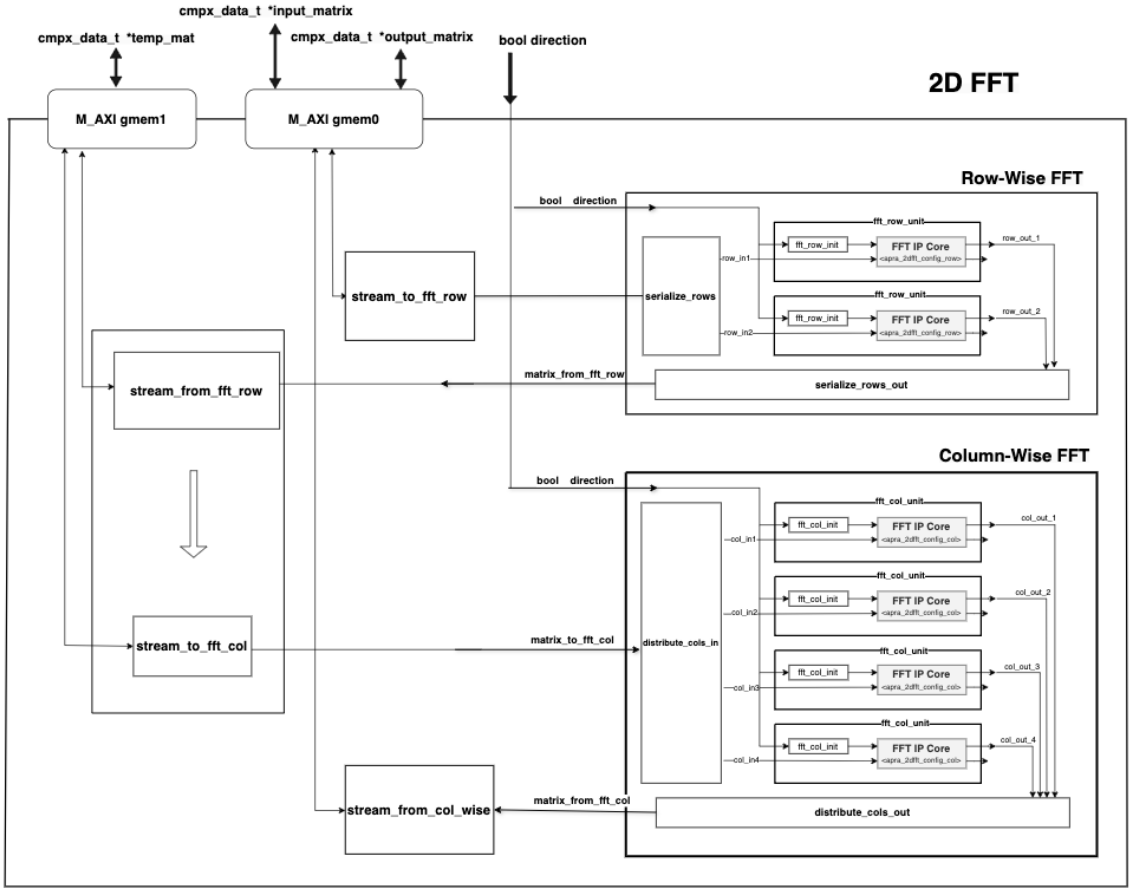


Figure 26. Architectural block diagram of 2D FFT Kernel.